CS 395 – Analysis of Algorithms

Chapter 3 – Brute Force and Exhaustive Search

Read 3.1-3.3

- Brute Force Algorithm Technique
  - Basic Idea
  - Brute Force Sorting algorithms
    - Selection Sort – Upcoming Weekly assignment
    - Bubble Sort
  - Polynomial Evaluation
  - Brute Force String Matching – Upcoming Weekly assignment
  - Brute-Force Closest-Pair

Brute Force: Basic Idea

A straightforward approach, usually based directly on the problem's statement and definitions of the concepts involved.

- *Force* – that of the computer.
- "just do it!"

Examples:

1. Computing a^n ($a > 0$, n a nonnegative integer)
2. Computing $n!$
3. Multiplying two matrices
4. Searching for a key of a given value in a list

Brute-Force Strengths and Weaknesses

- **Strengths**
  - wide applicability, simplicity
  - yields reasonable algorithms for some important problems (e.g., matrix multiplication, sorting, searching, string matching)
  - Standard algorithms for simple computation tasks, *i.e.*, sum and product of *n* numbers, finding maximum and minimum in a list, etc.
  - Sometimes useful for small input sizes
  - Can be used as a *yardstick*
- **Weaknesses**
  - rarely yields efficient algorithms
  - some brute-force algorithms are unacceptably slow
  - Not as constructive nor creative as other design techniques

3

Brute-Force Sorting Algorithm

Selection Sort:

Scan the array to find its smallest element and swap it with the first element. Then, starting with the second element, scan the elements to the right of it to find the smallest among them and swap it with the second elements.

Generally, on pass i ($0 \le i \le n$-2), find the smallest element in $A[i \ldots n\text{-}1]$ and swap it with $A[i]$

Selection Sort Example

i=	j=0	1	2	3	4	5	6
0	89	45	68	90	29	34	17
1	17	45	68	90	29	34	89
2	17	29	68	90	45	34	89
3	17	29	34	90	45	68	89
4	17	29	34	45	90	68	89
5	17	29	34	45	68	90	89
6	17	29	34	45	68	89	90

Selection Sort Pseudocode

ALGORITHM *SelectionSort*$(A[0..n-1])$

```
//Sorts a given array by selection sort
//Input: An array A[0..n − 1] of orderable elements
//Output: Array A[0..n − 1] sorted in ascending order
for i ← 0 to n − 2 do
    min ← i
    for j ← i + 1 to n − 1 do
        if A[j] < A[min]   min ← j
    swap A[i] and A[min]
```

Analysis of Selection Sort

Time efficiency

Number of comparisons:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2}.$$

Complexity of Selection Sort is $\Theta(n^2)$

Number of swaps: (n-1), so, $\Theta(n)$

Space efficiency: Θ (1)

Stability: maintains the relative order of records with equal keys (i.e. values). So, is a "stable" algorithm.

Brute-Force Sorting Algorithm

Bubble Sort:

Compare adjacent elements of an array (list) and swap them if they are out of order.

On the first pass, the largest element is "bubbled" up to the last position of the list.

The next pass bubbles up the second largest element, and so on until after *n*-1 passes, the list is sorted.

Bubble Sort Example

89	$\overset{?}{\leftrightarrow}$	45		68		90		29		34		17
45		89	$\overset{?}{\leftrightarrow}$	68		90		29		34		17
45		68		89	$\overset{?}{\leftrightarrow}$	90	$\overset{?}{\leftrightarrow}$	29		34		17
45		68		89		29		90	$\overset{?}{\leftrightarrow}$	34		17
45		68		89		29		34		90	$\overset{?}{\leftrightarrow}$	17
45		68		89		29		34		17	|	90
45	$\overset{?}{\leftrightarrow}$	68	$\overset{?}{\leftrightarrow}$	89	$\overset{?}{\leftrightarrow}$	29		34		17	|	90
45		68		29		89	$\overset{?}{\leftrightarrow}$	34		17	|	90
45		68		29		34		89	$\overset{?}{\leftrightarrow}$	17	|	90
45		68		29		34		17	|	89		90

etc.

Bubble Sort Pseudocode

ALGORITHM *BubbleSort*$(A[0..n-1])$

```
//Sorts a given array by bubble sort
//Input: An array A[0..n − 1] of orderable elements
//Output: Array A[0..n − 1] sorted in nondecreasing order
for i ← 0 to n − 2 do
    for j ← 0 to n − 2 − i do
        if A[j + 1] < A[j] swap A[j] and A[j + 1]
```

Analysis of Bubble Sort

Time efficiency:

Number of comparisons:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n-2-i) - 0 + 1]$$

$$= \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2).$$

Complexity of Bubble Sort is $\Theta(n^2)$

Number of swaps: depends on input.

Worst case is when the input array is in decreasing order.

$$S_{worst}(n) = C(n) = \frac{(n-1)n}{2} \in \Theta(n^2).$$

Space efficiency: Θ (1)

Stability: stable algorithm.

Brute-Force Polynomial Evaluation

Problem: Find the value of polynomial

$p(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x^1 + a_0$ **at a point** $x = x_0$

Brute-force algorithm

```
p ← 0.0
x ← x0
for i ← n downto 0 do // for each term
    power ← 1
        for j ← 1 to i do  //compute x^i
            power ← power * x
        p ← p + a[i] * power
return p
```

Efficiency: $\Theta(n^2)$

12

What is the value of this polynomial at x0?

Notice how many times we have to multiply power*x

Polynomial Evaluation: Improvement

We can do better by evaluating from right to left:

<u>Better brute-force algorithm</u>

$x \leftarrow x0$
$p \leftarrow a[0]$
$power \leftarrow 1$
for $i \leftarrow 1$ **to** n **do**
    $power \leftarrow power * x$
    $p \leftarrow p + a[i] * power$
return p

Efficiency: $\Theta(n)$

Brute-Force String Matching

- *pattern*: a string of *m* characters to search for
- *text*: a (longer) string of *n* characters to search in
- problem: find a substring in the text that matches the pattern

Brute-force algorithm

Step 1 Align pattern at beginning of text

Step 2 Moving from left to right, compare each character of pattern to the corresponding character in text until
- all characters are found to match (successful search); or
- a mismatch is detected

Step 3 While pattern is not found and the text is not yet exhausted, realign pattern one position to the right and repeat Step 2

Examples of Brute-Force String Matching

Pattern: 001011

Text: 10010101101001100101111010

001011

Pattern: happy

Text: It is never too late to have a happy childhood.

happy

Pseudocode

ALGORITHM *BruteForceStringMatch*$(T[0..n-1], P[0..m-1])$

```
//Implements brute-force string matching
//Input: An array T[0..n − 1] of n characters representing a text and
//        an array P[0..m − 1] of m characters representing a pattern
//Output: The index of the first character in the text that starts a
//        matching substring or −1 if the search is unsuccessful
for i ← 0 to n − m do
    j ← 0
    while j < m and P[j] = T[i + j] do
        j ← j + 1
    if j = m return i
return −1
```

Efficiency

Time Efficiency:

- **Best case:** $\Theta(m)$
- **Worst case:** $\Theta(n*m)$
- **Average case:** $\Theta(n+m) \approx \Theta(n)$

Space Efficiency: $\Theta(1)$

Closest-Pair Problem

Find the two closest points in a set of *n* points (in the two-dimensional Cartesian plane).

- **Assume for simplicity a two-dimensional space of points**
- **Assume each point is specified by their (*x*, *y*) Cartesian coordinates**
- **The distance between two points $P_i = (x_i, y_i)$ and $P_j = (x_j, y_j)$ is given by the Euclidean distance**

$$d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Brute-force algorithm

Compute the distance between every pair of distinct points and return the indexes of the points for which the distance is the smallest.

18

Draw example.

Closest-Pair Brute-Force Algorithm

ALGORITHM *BruteForceClosestPoints*(P)

```
//Input: A list P of n (n ≥ 2) points P_1 = (x_1, y_1), ..., P_n = (x_n, y_n)
//Output: Indices index1 and index2 of the closest pair of points
dmin ← ∞
for i ← 1 to n − 1 do
    for j ← i + 1 to n do
        d ← sqrt((x_i − x_j)^2 + (y_i − y_j)^2) //sqrt is the square root function
        if d < dmin
            dmin ← d; index1 ← i; index2 ← j
return index1, index2
```

Efficiency:

How to make it faster?

19

The basic operation of the algorithm is computing the Euclidean distance between two points. The square root is a complex operation who's result is often irrational, therefore the results can be found only approximately. Computing such operations are not trivial. One can avoid computing square roots by comparing distance squares instead.

Closest-Pair Brute-Force Algorithm (cont.)

Basic Operation: square root

But can do away with this.

Then "squaring" is the basic operation

Time Efficiency:

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 2 = 2\sum_{i=1}^{n-1} (n-i) = n(n-1) \in \Theta(n^2)$$

Space Efficiency: Θ (1)

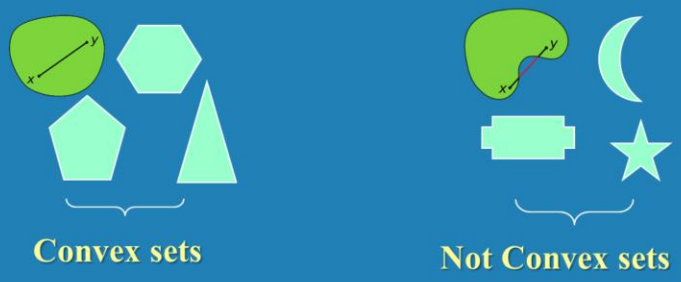
20

The basic operation of the algorithm is computing the Euclidean distance between two points. The square root is a complex operation who's result is often irrational, therefore the results can be found only approximately. Computing such operations are not trivial. One can avoid computing square roots by comparing distance squares instead.

Convex-Hull Problem

Definition - Convex

A set of points (finite or infinite) in the plane is called *convex* if for any two points *p* and *q* in the set, the entire line segment with the endpoints at *p* and *q* belong to the set.



Convex sets **Not Convex sets**

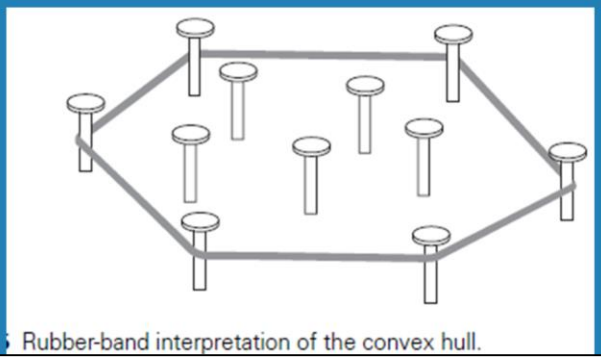
21

Draw example.

Convex-Hull

Formally: The *convex hull* of a set *S* of points is the smallest convex set containing *S*, *i.e.*, the convex hull of *S* is a subset of any convex set containing *S*.

Informally: It's a rubber-band wrapped around the "outside" points.



Rubber-band interpretation of the convex hull.

Draw example.